

Report: GSM Network in a Box

June 19, 2015

Performed by:	Fraunhofer Portugal, Hochschule Bonn-Rhein-Sieg
Time period	01. Sep 2014 - 31 June 2015
Persons:	B.Sc. Markus Kessel, M.Sc. Michael Rademacher

(This deliverable is in the framework of Project ICT4DCC (ICT4D Competence Center), with reference “Projeto n.º NORTE-07-0124-FEDER-000042”, co-financed by the Regional Operational Program of the North (ON.2 – The New North) [“Programa Operacional Regional do Norte (ON.2 – O Novo Norte)”], in accordance with the National Strategic Reference Framework [“Quadro de Referência Estratégico Nacional (QREN)”], through the European Regional Development Fund [“Fundo Europeu de Desenvolvimento Regional (FEDER)”].



O NOVO NORTE
PROGRAMA OPERACIONAL
REGIONAL DO NORTE



UNIÃO EUROPEIA
Fundo Europeu de
Desenvolvimento Regional

Abstract

This project report describes the development process of a small, cost-effective communication solution as an alternative to a large scale and expensive GSM-based network provided by a professional carrier or ISP.

In order to make this solution available and preferable for rural areas the expense factor needs to be reduced as much as possible. Therefore several considerations need to be taken into account. To reduce the CAPEX the system is based on off-the-shelf hardware components and implements already existing open-source software solutions in combination to enable the respective functionalities. Due to the usage of standard hardware and software the installation, operation and maintenance of the system can be performed even by personnel without special education. Thereby the OPEX of the system can be lowered.

In order to make the different services and functionalities available to almost all users and customers a clear website is presented with the key features of this system. This aspect is supported furthermore by the instructional purpose and structure of this report.

The initial setup consists of a Linux based system in combination with an *ip.access* nanoBTS base station. Various software packages are used to establish the functionalities of the solution. The most packages originate from the *Osmocom* OpenBSC project and need additional packages like OpenGGSN for the support of data communications based on GPRS. Besides of these packages the installation and configuration of a Linux Call Router, an Asterisk server and the management web interface is also needed. After the installation of these various components the call forwarding procedures are defined in the Linux Call Router and several configuration files for the OpenBSC components are changed as well as the configuration of the nanoBTS.

With these configurations it is possible to provide a local GSM network and allow clients to place and receive calls from other subscribers connected to this base station.

The next step enables the system to connect mobile clients like cellphones with stationary softphones that are registered at the local Asterisk server. Therefore a dialplan and various extensions are needed.

In combination with an Internet connection like it can be established with the WiBACK system at a low cost the systems is capable of even more functionalities. Considering that a sipgate account can be used to realise the interconnection to an existing public telecommunication network and even to other cellular network provided by professional carriers. To realise this a few changes to the configurations of the Asterisk server and the Linux Call Router need to be added and a valid sipgate account need to be provided.

In conclusion various tests are conducted to ensure the correct functionality of the system and to provide a profound statement regarding the reliability of the solution and maybe to pinpoint the possible need for improvement.

Contents

List of Tables	iv
1 Introduction	2
2 GSM network with local operation scope	3
2.1 Features	3
2.2 Description and Installation of the required components	4
2.2.1 Base system	4
2.2.2 OpenBSC with [E]GPRS	5
2.2.3 mISDN and mISDN-user	8
2.2.4 LCR (Linux Call Router)	8
2.2.5 Asterisk	9
2.2.6 OpenBSC and OsmoSGSN HLR Management Interface	9
2.3 Configuration of the required components	9
2.3.1 LCR	10
2.3.2 Asterisk	11
2.3.3 OpenBSC	12
2.4 osmo-sgsn	15
2.5 OpenGGSN	16
2.6 nanoBTS	17
2.7 Intermediate summary	17
3 Concept of wide-area GSM Network in a Box (NiaB) with public connectivity	18
3.1 Internet interconnect using WiBACK	18
3.2 Public connectivity	19
3.2.1 LCR configuration	19
3.2.2 Asterisk configuration	19
3.2.3 Sipgate configuration	21
4 Evaluation in a lab environment	24
4.1 Compatible Clients	25
4.2 Number of simultaneously placed calls	25
4.3 Call setup success rate	26
4.3.1 Softphone caller setup	26
4.3.2 Cellphone callee setup	26
4.3.3 Cellphone caller setup	27
4.3.4 Positions	27
4.3.5 Results	28
5 Conclusion	31
5.1 Future Work	31
References	32

List of Tables

1	List of tested Clients in the laboratory environment	25
2	Results of the conducted experiments between a softphone and a cellphone .	29
3	Results of the conducted experiments between two cellphones	30

List of Acronyms

AAA Authentication Authorization Accounting
AUC Authentication Center
BSC Base Station Controller
CAPEX capital expenditure
GSM Global System for Mobile Communications
HLR Home Location Register
ISP Internet Service Provider
MSC Mobile-services Switching Centre
NiaB Network in a Box
OPEX operational expenditure
SMSC Short Message Service Centre
VoIP Voice over Internet Protocol
VoIP Voice-over-IP

1 Introduction

While Global System for Mobile Communications (GSM) networks are widely available in cities and dense areas a deployment in rural areas is still a challenging tasks for Internet Service Providers (ISPs) due to the lack of profit. Currently available base stations are expensive and need special mounting places to serve a large area. To amortize the cost of such a base station a certain number of customers is required which is challenging in rural areas where the population density is usually low.

A different concept will be discussed in this report. Instead of large cells serving a large area a concept of small femto-cells will be presented. These femto-cells are mobile and can be placed near a village to provide coverage to a certain amount of customers. By using off-the-shelf hardware and open-source software components the capital expenditure (CAPEX) in this solution decreases compared to industrial grade GSM solutions and is available also for small ISP. In addition to that, this project aims at the development of a fast and easy to deploy GSM network. The goal is to provide non-technicians with the possibility to set-up the network without special training even in a challenging environment. This greatly reduces the operational expenditure (OPEX) of GSM network.

This report is structured into three main parts.

In section 2 the concept, installation (section 2.2) and configuration (section 2.3) of a GSM NiaB with local operation scope will be described.

In section 3 the developed solution will be extended for a possible interconnection with the public telecommunication network. This section contains the description of using the GSM NiaB solution together with WiBACK. A low-cost self-managing wireless Backhaul solution to provide the connection to rural areas where the GSM should operate [Fok15].

A comprehensive evaluation of the developed solution is presented in section 4. This evaluation includes a list of compatible clients as well as two important stability test. First the number of possible simultaneously placed calls is tested. Afterwards a call setup success rate is determined which is an important parameter for the Quality of Experience in a GSM Network.

The report closes with a conclusion in section 5.

2 GSM network with local operation scope

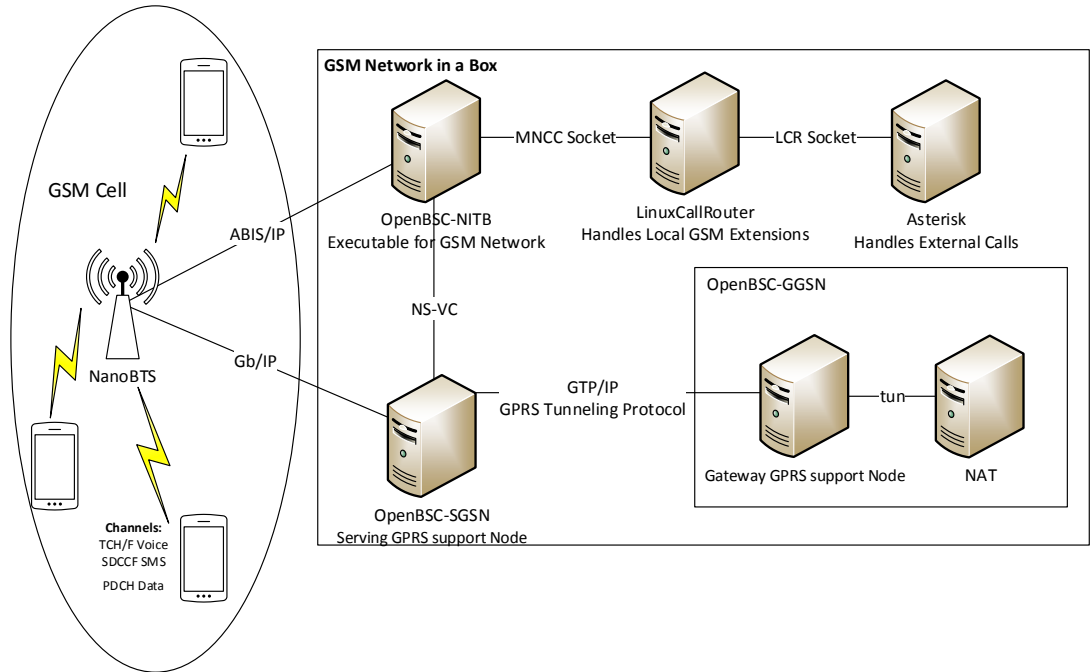


Figure 2.1: Concept of a GSM NiaB with local operation.

For the operation of the GSM NiaB the following services are needed. They can be combined on a single physical or virtual machine as shown in figure 2.1:

- OpenBSC-NITB [WF13a]: The executable program of the OpenBSC project.
- Linux Call Router (LCR) [Eve14]: A software based private branch exchange. It provides intercommunication between the local assigned GSM user terminals in the cell.
- Asterisk: An open source telephony switching and private branch exchange service for Linux. It handles possible external calls.
- OpenBSC-SGSN [WF13a]: The OpenBSC Implementation of a Serving GPRS support node.
- OpenBSC-GGSN [WF13a]: The OpenBSC Implementation of a Gateway GRPS support node.
- ip.access nanoBTS [WF13a]: Commercial hardware for the air interface.

2.1 Features

The intended GSM NiaB will provide the following features:

- Local GSM extensions managed by the LCR or Asterisk.

- Calls between GSM subscribers.
- SMS can be sent from console and between registered phones.
- Per BTS capacity (8 Timeslots):
 - Timeslot 0 allocated for control and SMS traffic: 4 SMS sent simultaneously, 20 SMS queue.
 - Timeslots 1-3 allocated for voice calls: 3 simultaneous GSM calls per nanoBTS.
 - Timeslots 4-7 allocated for data traffic[WF13b]: 211 kbit/s max. download speed, 84 kbit/s max. upload speed.
- Handover between nanoBTS controlled by same Base Station Controller (BSC) with multiplied licensed frequency each assigned to a different nanoBTS.

2.2 Description and Installation of the required components

As shown in figure 2.1 the NiaB consists of multiple software components relying on each other. In the following the installation and configuration of these components will be described. The purpose of this extended section is to provide the reader with the possibility to set up a GSM NiaB from scratch and to understand the main software involved. A good understanding of Linux based operating system and knowledge about the modular concept of GSM is expected.

2.2.1 Base system

The operating system for NiaB approach is Debian in its latest version 7. The installation of a graphical user interface is not required so a basic installation of the Linux system environment is sufficient in this context. All further installation steps and commands should be conducted via Secure Shell (ssh). The installation of a Debian system is not covered in this report since there are numerous reference dealing with this topic [tea13].

After the installation of the base system the following packages need to be installed for further processing and building of the software components.

Listing 1: Install: Dependencies

```
1 apt-get -y -qq install libdbi0-dev libdbd-sqlite3 build-essential libtool autoconf
  automake git-core pkg-config make libortp-dev libgsm1-dev asterisk asterisk-dev
  libssl-dev ncurses-base libncurses5-dev libncursesw5-dev python-setuptools
  linux-headers-$(uname -r)
```

In addition to that the following variables are used in the following subsection for convenience of the reader:

Listing 2: Install: Dependencies

```
1 BUILDDIR='/home/$user/gsmtestbed' # The build directory
2 LOG_FILE='/home/$user/gsmtestbed'"/deploy_gsmtestbed.log"
3 NUM_CPU='nproc'
```

2.2.2 OpenBSC with [E]GPRS

This section documents how to install the osmo-nitb [Wel13] software. A stand-alone GSM NiaB program implementing a minimal subset of programs needed for the operation of a GSM network.

Please note that osmo-bsc alone is not sufficient for the needs of the described project. Osmo-bsc is a version of OpenBSC which runs in pure BSC-only mode, i.e. it will implement the BSC functionality and the Abis and air interfaces, but not emulate a Mobile-services Switching Centre (MSC), Short Message Service Centre (SMSC), Home Location Register (HLR) and Authentication Center (AUC) like the osmo-nitb (network in the box) version of OpenBSC. If GPRS/EDGE features are desired, OpenGGSN must first be installed before compiling OpenBSC. Overall the installation of osmo-nitb is broken down into 5 sections:

- OpenGGSN (for GPRS support)
- libosmocore (OpenBSC libraries)
- libosmo-abis (Abis over IP libraries)
- OpenBSC (openBSC executables and ipaccess tools)
- libtelnet (ipaccess telnet client)

The five sections will be covered in the following. They include all needed Linux terminal commands to build and install the components.

OpenGGSN In order to enable GPRS/EDGE features, the modules osmo-sgsn and OpenGGSN need to be installed. There exists a need to (re-)build OpenBSC after having installed OpenGGSN so that the libgtp dependency is detected and the SGSN binary (osmo-sgsn) is built.

Listing 3: Build and install: OpenGGSN

```
1 # Building: OpenGGSN
2 cd $BUILDDIR
3 git clone -q git://git.osmocom.org/openggsn.git
4 cd $BUILDDIR/openggsn
5 git checkout -q 0527289c2be316f07166e850ab92bb2dd822a7de
6 autoreconf -fi &>> $LOG_FILE
7 automake --add-missing
8 ./configure --prefix=/usr/local &>> $LOG_FILE
9 make -s -j$NUM_CPU &>> $LOG_FILE
10 # Installing OpenGGSN
11 make -s install &>> $LOG_FILE
```

```
12 mkdir -p /etc/ggsn
13 cp examples/ggsn.conf /etc/ggsn.conf
14 cp debian/openggsn.init /etc/init.d/openggsn
15 update-rc.d openggsn defaults
```

libosmocore (OpenBSC libraries) The following commands build and install the OpenBSC libraries:

Listing 4: Build and install: OpenBSC

```
1 # Building: libosmocore"
2 cd $BUILDDIR
3 git clone -q git://git.osmocom.org/libosmocore.git
4 cd $BUILDDIR/libosmocore
5 git checkout -q 25bcb6cecef0af52599c6d220e226f937ab78b12
6 autoreconf -fi &>> $LOG_FILE
7 ./configure &>> $LOG_FILE
8 make -s -j$NUM_CPU &>> $LOG_FILE
9 # Installing: libosmocore"
10 make -s install &>> $LOG_FILE
```

These libraries provide the core functionalities for the GSM NiaB approach.

libosmo-abis (Abis over IP libraries) The following commands build and install libosmo-abis. These libraries are needed for the emulation of an Abis interface:

Listing 5: Build and install: libosmo-abis

```
1 #Building: libosmo-abis"
2 cd $BUILDDIR
3 git clone -q git://git.osmocom.org/libosmo-abis.git
4 cd $BUILDDIR/libosmo-abis
5 git checkout -q bd3f854f3c2d468722b26289fa52e381da41a455
6 autoreconf -fi &>> $LOG_FILE
7 ./configure &>> $LOG_FILE
8 make -s -j$NUM_CPU &>> $LOG_FILE
9 #Installing: libosmo-abis
10 make -s install &>> $LOG_FILE
11 ldconfig
```

OpenBSC (OpenBSC executables and ip.access tools) In this section the OpenBSC executables and the needed ipaccess tools will be covered. The ipaccess tools are important for the communication with the nanoBTS.

Since it is unlikely that SIM cards will be produced in this approach a patch that blocks roaming users from accessing the GPRS data network needs to be undone. See [Wel10a] for more information. This is needed to provide users with the possibility to use the SIM cards of their current operator in the local operation scope of the network.

First, the OpenBSC repository needs to be downloaded:

Listing 6: Download: OpenBSC

```
1 cd $BUILDDIR
2 git clone -q git://git.osmocom.org/openbsc.git
3 cd $BUILDDIR/openbsc/openbsc
4 git checkout -q 75e13a41dc5ebd1dfd009814a60cf16df8681e74
```

Afterwards, the following commands reverse the original patch where OpenBSC denies data access to SIM cards that do not have the same MCC and MNC as the intended network:

Listing 7: Apply SIM card patch

```
1 wget -q http://cgit.osmocom.org/openbsc/patch/?id=
    eafe22ca7290280fca0f8d31d70239fd0b0dbb9d -O gprs_roaming.patch
2 patch -s -p2 -RN -i gprs_roaming.patch
```

And finally the openbsc programs can be build and installed:

Listing 8: Build and install: OpenBSC

```
1 autoreconf -fi &>> $LOG_FILE
2 export PKG_CONFIG_PATH=/usr/local/lib/pkgconfig
3 ./configure &>> $LOG_FILE
4 make -s -j$NUM_CPU &>> $LOG_FILE
5 #Installing: OpenBSC & OsmoSGSN
6 make -s install &>> $LOG_FILE
7 mkdir -p /etc/openbsc
8 cp doc/examples/osmo-sgsn/osmo-sgsn.cfg /etc/openbsc/
9 cp ../debian/osmocom-nitb.default /etc/default/osmocom-nitb
10 cp ../debian/osmocom-sgsn.default /etc/default/osmocom-sgsn
11 cp ../debian/osmocom-nitb.init /etc/init.d/osmocom-nitb
12 cp ../debian/osmocom-sgsn.init /etc/init.d/osmocom-sgsn
13 sed "s/osmocom/openbsc/" -i /etc/default/osmocom-sgsn
14 sed "s/osmocom/openbsc/g" -i /etc/default/osmocom-nitb
15 sed "s/var/lib/etc/" -i /etc/default/osmocom-nitb
16 sed "s/osmo-nitb/openbsc/" -i /etc/default/osmocom-nitb
17 echo "DAEMON_ARGS=\"-m -P -C\"" >> /etc/default/osmocom-nitb
18 sed "s/DAEMON=/usr/bin/osmo-sgsn/DAEMON=/usr/local/bin/osmo-sgsn/" -i /etc/
    init.d/osmocom-sgsn
19 sed "s/DAEMON=/usr/bin/osmo-nitb/DAEMON=/usr/local/bin/osmo-nitb/" -i /etc/
    init.d/osmocom-nitb
20 update-rc.d osmocom-sgsn defaults
21 update-rc.d osmocom-nitb defaults
```

ipaccess telnet client The following commands build and install the ip.access telnet client:

Listing 9: Build and install: ipaccess telnet client

```
1 # Building: ipaccess telnet client"
2 cd $BUILDDIR
3 git clone -q git://git.osmocom.org/libtelnet.git
4 cd $BUILDDIR/libtelnet
5 git checkout -q 1465a8668ddb925494d5a1ef5adffc864a924459
6 autoreconf -fi &>> $LOG_FILE
7 ./configure &>> $LOG_FILE
8 make -s -j$NUM_CPU &>> $LOG_FILE
```

```
9 # Installing: ipaccess telnet client"
10 ldconfig
```

2.2.3 mISDN and mISDN-user

The following commands build and install the mISDN Kernel module.

Listing 10: Build and install: mISDN Kernel module

```
1 # Building: mISDN Kernel Module
2 cd $BUILDDIR
3 git clone -q git://git.misdn.eu/mISDN.git/
4 cd $BUILDDIR/mISDN
5 ./configure &>> $LOG_FILE
6 make -s -j$NUM_CPU modules &>> $LOG_FILE
7 make -s modules_install &>> $LOG_FILE
8 ./std2kern
9 kernel_version=`uname -r`
10 ln -sf /lib/modules/$kernel_version/extra/mISDN_core.ko /lib/modules/
    $kernel_version/kernel/drivers/isdn/mISDN/mISDN_core.ko
11 ldconfig
12 depmod -a
13 modprobe mISDN_l1loop pri=1 nchannel=30 interfaces=2
14 echo "mISDN_core" >> /etc/modules
15 echo "mISDN_dsp" >> /etc/modules
16 echo "mISDN_l1loop pri=1 nchannel=30 interfaces=2" >> /etc/modules
```

At this point two mISDN loopback devices are installed. Using the configuration in `/etc/modules` they are loaded at every boot [Wel10b]. If mISDN l1loop does not load because of an error similar to this warning "mISDN l1loop: disagrees about version of symbol" the module was not installed properly with the above procedure.

Besides the specific mISDN kernel module the mISDN-user executable is needed

Listing 11: Build and install: mISDN-user

```
1 # Building: mISDN-user programm
2 cd $BUILDDIR
3 git clone -q git://git.misdn.eu/mISDNuser.git/
4 cd $BUILDDIR/mISDNuser
5 git checkout -q 1dc4c24d27ed9c46e2fdbf7fb4b178e549a58e8d
6 make -s -j$NUM_CPU &>> $LOG_FILE
7 ./configure &>> $LOG_FILE
8 make -s -j$NUM_CPU &>> $LOG_FILE
9 make -s install &>> $LOG_FILE
10 ldconfig
```

2.2.4 LCR (Linux Call Router)

Listing 12: Build and install: LCR(Linux Call Router)

```
1 # Building: LCR
2 cd $BUILDDIR
3 git clone -q git://git.misdn.eu/lcr.git/
4 cd $BUILDDIR/lcr
```

```
5 | git checkout -q 0f0568583f9a4c91de2c5289e8556d857260093c
6 | ln -sf /usr/lib/i386-linux-gnu/libgsm* /usr/lib/
7 | ln -s $BUILDDIR/libosmocore/ .
8 | ln -s $BUILDDIR/openbsc/openbsc/ .
9 | ./autogen.sh &>> $LOG_FILE
10 | ./configure --with-asterisk --with-gsm-bs LDFLAGS="-L/usr/local/lib" &>> $LOG_FILE
11 | make -s -j$NUM_CPU &>> $LOG_FILE
12 | make -s install &>> $LOG_FILE
13 | ldconfig
14 | cp debian/lcr.default /etc/default/lcr
15 | cp debian/lcr.init /etc/init.d/lcr
16 | chmod +x /etc/init.d/lcr
17 | sed "s/DAEMON=/usr/sbin/\$NAME/DAEMON=/usr/local/sbin/\$NAME/" -i /etc/init.
    d/lcr
18 | sed "s/RUNLCR=no/RUNLCR=yes/" -i /etc/default/lcr
```

2.2.5 Asterisk

Fortunately, there is no customized build of asterisk necessary for managing calls outside of the LCR context. It is possible to install the latest available version of Asterisk for the Debian distribution.

Listing 13: Install: Asterisk

```
1 | apt-get install asterisk
```

2.2.6 OpenBSC and OsmoSGSN HLR Management Interface

To easily administer the OpenBSC and its accompanying services a custom web management interface has been developed at the Fraunhofer Institute Fokus. The code of this project has been included in the official osmocom repository. Instead of using an internal version the following assumes an installation directly from osmocom. For additional information read the README.txt file included in the root directory of the git checkout. Also be sure to customize the appropriate files to the current needs of the project.

Listing 14: Build and install: Management interface

```
1 | cd $BUILDDIR
2 | git clone -q git://git.osmocom.org/python/osmo-oohmi
3 | cd $BUILDDIR/osmo-oohmi
4 | sed "s/<<Provider_Name>>/\$PROVIDER/" -i hlr_mgmt/__init__.py
5 | python setup.py install
6 | cp production.ini /etc/openbsc/production.ini
7 | cp osmocom-oohmi /etc/init.d/osmocom-oohmi
8 | update-rc.d osmocom-oohmi defaults
```

2.3 Configuration of the required components

Besides the installation of the required components a significant amount of configuration is needed to provide an interconnection between the various programs and modules. Since

almost all components are non-industrial grade and open-source the documentation is usually provided inside the source code.

2.3.1 LCR

The most configuration work needs to be done for the Linux Call Router. Basically the LCR serves as bridge between the OpenBSC process and the Asterisk. First the extensions need to be generated which provides the internal (Asterisk) and external (OpenBSC) number in the context of GSM. With the following commands 30.000 different numbers will be generated which can be used by the customers.

Listing 15: Generation of LCR extensions for mobile phones

```
1 #!/bin/bash
2 for i in {20000..49999}; # This represents the tel. number
3 do
4     genextension $i GSM $i &>> $LOG_FILE
5 done
6 rm /usr/local/var/lib/lcr/extensions/*/phonebook
7 rm /usr/local/var/lib/lcr/extensions/*/secrets
```

The amount of extension as well as the style and length of the numbers can be adapted to the specific needs of the project.

An important step to ensure security of the system is to prevent executing the LCR with root privileges on the system. Instead it will be executed using the Asterisk user and group generated during the installation. This privileges configuration will be conducted in the `/usr/local/etc/lcr/options.conf`. The contemplated file should include the following lines

Listing 16: `/usr/local/etc/lcr/options.conf`

```
1 socketuser asterisk
2 socketgroup asterisk
```

For general configuration of the LCR only two sections are needed. These two sections describe the services the LCR process is connected to. In this project this the `gsm-bs` as well as the Asterisk ¹ The `gsm-bs` is an interface from the LCR to a GSM base station. In this project this base station is the `osmo-nitb` process. By using the `EXT` section the LCR will connect to the asterisk process using the `chan_lcr` interface. While configuring Asterisk the context `btsctrl` will be used to identify the GSM extension and the connection to the LCR. All this configuration will take place in the `/usr/local/etc/lcr/interface.conf` file and the following lines need to be included:

Listing 17: Config: LCR `/usr/local/etc/lcr/interface.conf`

```
1 # GSM network interface
2 [GSM]
3 gsm-bs
4 tones yes
```

¹Compare the installation of LCR where exactly these two external interfaces were provided in the configuration.

```
5 | earlyb no
6 | extension
7 | # Use chan_lcr (Asterisk PBX interface) as external interface.
8 | [Ext]
9 | remote asterisk
10 | context btsctrl
11 | extern
12 | earlyb yes
13 | tones no
```

After the general configuration of the LCR connections, the routing needs to be configured in the appropriate file `"/usr/local/etc/lcr/routing.conf"`. Again two different sections needs to be generated. The main section is responsible for receiving calls from the GSM interface configured in the previous step. The gexten sections is a custom routing name for identification of local subscribers and forwarding everything else to the external interface which is the remote Asterisk application. In this configuration the LCR is routing the internal calls without the Asterisk application.

Listing 18: Config: LCR `/usr/local/etc/lcr/routing.conf`

```
1 | [main]
2 | interface=GSM : goto ruleset=gexten
3 | #openbsc cellphone extensions are automatically assigned between 20000-49999
4 | [gexten]
5 | dialing=20000-49999: intern connect type=subscriber capability=digital-unrestricted
6 | default : extern
7 | : disconnect cause=1 display="Invalid Code"
```

2.3.2 Asterisk

Since there are numerous tutorials for the Asterisk PBC available this section will only cover the basic aspects for the intended operation. It needs to be ensured that the Asterisk knows about the `chan_lcr` by loading it in the `modules.conf`.

Listing 19: Config: Asterisk `/etc/asterisk/modules.conf`

```
1 | load => chan_lcr.so
```

Afterwards a basic dialplan will be generated. At the moment this dialplan has nearly no influence to the operation of the network.

Listing 20: Config: Asterisk `/etc/asterisk/extension.conf`

```
1 | [btsctrl]
2 | include => to_cellphones
3 | include => to_intern
4 | include => to_extern
5 |
6 | [to_cellphones]
7 | exten => _1X,1,GotoIf(\${\${CALLERID(name)}} != \"\"?4)
8 | exten => _1X,2,Set(CALLIDORIG=\${CALLERID(num)})
9 | exten => _1X,3,Set(CALLERID(num)=02\${CALLIDORIG})
10 | exten => _1X,4,Dial(LCR/GSM/\${EXTEN},120)
```


Not internal recognized calls get routed to the Asterisk by the LCR and are marked in the `btscrl` context. The "to cellphones" context can reroute them to the GSM network and also take calls from outside (for example fixed-line networks). Two additional dummy contexts were added. One is intended for a routing to additional local phones in the network for example SIP clients in a local office. The other is designed for an external routing to a public network.

2.3.3 OpenBSC

The next component of the system which needs to be configured is the OpenBSC itself. The most important configuration will be conducted in the `/etc/openbsc/openbsc.cfg` file and is listed below. For simplicity, explanations of the configuration file have been split up into two sections: network and BTS.

Listing 21: Config: OpenBSC `/etc/openbsc/openbsc.cfg` network section

```
1  !
2  ! OpenBSC configuration saved from vty
3  !  !
4  password foo
5  !
6  line vty
7    no login
8  !
9  log file /var/log/openbsc
10 e1_input
11   e1_line 0 driver ipa
12 network
13   network country code 262
14   mobile network code 23
15   short name rural
16   long name rural-connect
17   auth policy closed
18   location updating reject cause 13
19   encryption a5 1
20   neci 1
21   rrlp mode none
22   mm info 1
23   handover 1
24   handover window rxlev averaging 10
25   handover window rxqual averaging 1
26   handover window rxlev neighbor averaging 10
27   handover power budget interval 6
28   handover power budget hysteresis 3
29   handover maximum distance 3
30   timer t3101 10
31   timer t3103 0
32   timer t3105 0
33   timer t3107 0
34   timer t3109 0
35   timer t3111 0
36   timer t3113 60
37   timer t3115 0
38   timer t3117 0
39   timer t3119 0
40   timer t3141 0
```

Several aspects of this configuration need further explanation. The "network country code" must match to the country where the operation takes place. For Germany, this value is 262 and is related to the "mobile network code" which must match your operator MNC code. For a test network this is the value 23. To provide a nice name for the customers "short name" and "long name" set the network names to be used in the GSM 04.08 MM INFO message. An important parameter is the so called "auth policy". It defines and limits IMSI access to the network. It can take up 1 of 3 forms:

- closed: Don't allow anyone who is not marked as authorized=1 in the HLR database
- accept-all: Accept everyone into the network
- token: Uses a per-user TokenAuthPolicy

The "encryption" tells the BSC which algorithm to use to encrypt the air interface. Three different values are possible:

- A5/0 indicates no encryption
- A5/1 is encryption with phones that support it
- a5/2 is unsecure and should not be used anymore.

The handover parameters enables a handover between BTS nodes which share the same BSC. The handover maximum distance parameter is set to 3 for a specific configuration in order to simulate a handover scenario. This is necessary when nanoBTS antennas are too close for a real world scenario. This value can be increased up to 9999 if the base-stations have a larger distance (several hundred meter) in between. This value is highly dependable on the current deployment scenario, however, handover parameters can be fine-tuned by debugging osmo-nitb with the -d DHO flag.

After completing the network configuration above, we need to add specific information for each one of our nanoBTS antennas. Below is a sample configuration with some explanations. Please note that the following is for nanoBTS antennas, for different antenna types, certain configurations may change.

Listing 22: Config: OpenBSC /etc/openbsc/openbsc.cfg Network section

```
1  '''bts 0'''
2      '''type nanobts'''
3      '''description nanoBTS'''
4      '''band DCS1800'''
5      '''cell_identity 1'''
6      '''location_area_code 1'''
7      training_sequence_code 7
8      '''base_station_id_code 60'''
9      ms max power 15
10     cell reselection hysteresis 4
11     rxlev access min 0
12     channel allocator ascending
13     rach tx integer 9
14     rach max transmission 7
15     '''ip.access unit_id 1801 0'''
16     oml ip.access stream_id 255 line 0
```

```
17  '''gprs mode egprs'''
18  '''gprs routing area 1'''
19  '''gprs cell bvc1 1'''
20  '''gprs nsei 101'''
21  '''gprs nsvc 0 nsvci 101'''
22  gprs nsvc 0 local udp port 23000
23  gprs nsvc 0 remote udp port 23000
24  '''gprs nsvc 0 remote ip 10.30.0.110'''
25  trx 0
26    rf_locked 0
27    '''arfcn 871'''
28    nominal power 23
29    max_power_red 20
30    rsl e1 tei 0
31    timeslot 0
32      phys_chan_config CCCH+SDCCH4
33    timeslot 1
34      phys_chan_config SDCCH8
35    timeslot 2
36      phys_chan_config TCH/F
37    timeslot 3
38      phys_chan_config TCH/F
39    timeslot 4
40      phys_chan_config PDCH
41    timeslot 5
42      phys_chan_config PDCH
43    timeslot 6
44      phys_chan_config PDCH
45    timeslot 7
46      phys_chan_config PDCH
```

The "bts <id>" (first line) is a sequential identifier used by the BSC. This value must start at 0 for the first BTS, and increase gradually for each new BTS. The "type" indicates the type of BTS antenna. In our case, we tell the BSC we will use nanoBTS as antennas. Supported types are at the moment bs-11 and nanobts.

The "description" is an optional parameter for human readable identification of the BTS. The "cell_identity" is unique to each BTS. The "ip.access unit_id" corresponds to the unit ID configured on the nanoBTS. The "gprs mode" tells the BTS to operate as gsm, gprs or egprs. This had no configuration impact on the remaining gprs parameters. The "routing area, cell bvc1, nsei" and "nsvi" are unique to each BTS as well. The "gprs nsvc 0 remote ip" entry is the IP of the "OpenBSC (NITB) Server". Note that the nanoBTS only acts as DHCP client which means that a DHCP server needs be running inside the network. This DHCP server needs to assign addresses in the same subnet as the OpenBSC NITB Server.

The "timeslot" and "phys_chan_config" indicate how to use the channels within the BTS. See nanoBTS datasheet for all configurations supported by nanoBTS

- NONE: do not use this timeslot
- CCCH: regular CCCH configuration (BCCH+RACH+PCH+AGCH)
- CCCH+SDCCH4: CCCH plus SDCCH/4 combination

- TCH/F: Full TCH
- TCH/H: Half TCH
- SDCCH8: SDCCH/8 combination
- PDCH: Packed Data CHannel
- TCH/F_PDCH: Alternating TCH/F and PDCH use (does not work yet)

2.4 osmo-sgsn

This section documents the configuration of the "Serving GPRS support node (SGSN)" function. First a virtual interface in parallel to the main network interface of Debian has to be created. This can be conducted with the following command:

Listing 23: Config: Virtual interface

```
1 ifconfig eth0:0 MAIN-IP+1 netmask 255.255.255.0
```

A sample configuration of the SGSN will be provided in the following.

Listing 24: Config: SGSN /etc/openbsc/osmo-sgsn.cfg

```
1 !
2 ! Osmocom SGSN configuration
3 !
4 !
5 line vty
6 no login
7 !
8 sgsn
9   '''gtp local-ip IP'''
10   '''ggsn 0 remote-ip MAIN-IP+2'''
11   ggsn 0 gtp-version 1
12 !
13 ns
14   timer tns-block 3
15   timer tns-block-retries 3
16   timer tns-reset 3
17   timer tns-reset-retries 3
18   timer tns-test 30
19   timer tns-alive 3
20   timer tns-alive-retries 10
21   '''encapsulation udp local-ip 10.30.0.110'''
22   encapsulation udp local-port 23000
23   encapsulation framerelay-gre enabled 0
24 !
25 bssgp
26 !
```

The most important parameters will be further described:

- The "encapsulation udp local-ip" entry is the IP of the "OpenBSC (NITB) Server"
- The "gtp local-ip" entry is the IP of the "SGSN Server".

- The "ggsn 0 remote-ip" entry is the IP of the "GGSN Server".
- The encapsulation settings must be the same IP/port than the defined setup in "/etc/openbsc/openbsc.cfg".

2.5 OpenGGSN

The "Gateway GPRS support node (GGSN)" function is relatively easy to configure. Only a few parameters specific to the setup need to be defined and the preparation of the network interfaces for forwarding data correctly. The /etc/ggsn.conf is pretty well documented. The most interesting part is the configuration of the GTP link which must match the "ggsn 0 remote-ip" entry in osmo-sgsn.cfg. This is the main IP address of the GGSN Server.

Listing 25: Config: GGSN /etc/ggsn.conf

```
1 listen MAIN-IP+2
2 net 192.168.254.0/24
3 dynip 192.168.254.0/24
4 pcodns1 primary-DNS-IP
5 pcodns2 8.8.8.8
```

To ensure proper routing a script needs to be generated which will be executed after the interface has been brought up.

Listing 26: Config: GGSN interfaces /etc/ggsn/ip-up

```
1 #!/bin/bash
2 route del -net 192.168.254.0 netmask 255.255.255.0
3 route add -net 192.168.254.0 netmask 255.255.255.0 gw $2
```

The last important configuration is to ensure that the OS knows about the interface by adding it to the network interfaces configuration file:

Listing 27: Config: /etc/network/interfaces

```
1 auto eth0:2
2 iface eth0:2 inet static
3     address <MAIN-IP+2>
4     netmask 255.255.255.0
5     gateway <IP address of the gateway>
```

or by entering the following command line:

Listing 28: Config: Interface via CLI

```
1 ifconfig eth0:2 <IP-MAIN+2> netmask 255.255.255.0
```

Additionally "ip_forward" should be enabled and the packets should be masqueraded on the main interface using the following two commands:

Listing 29: Config: Routing

```
1 $ echo 1 > /proc/sys/net/ipv4/ip_forward
2 $ iptables -A POSTROUTING -s 192.168.254.0/24 -t nat -o eth0 -j MASQUERADE
```

2.6 nanoBTS

Libosmcore installs a set of tools for configuring your nanoBTS. For this project it is only needed to set the unit ID and the OML address. Optionally, telnet access can also be enabled. After unpacking the nanoBTS (or if this value should to be changed later) the unit ID has to be set. This unit ID is (usually) a 4 digit value used by the OpenBSC server to identify and control the nanoBTS. This can be set with the following command:

Listing 30: Config: nanoBTS, set Unit-ID

```
1 ipaccess-config -u <Unit_ID>/0/0 -r <BTS_IP>;
```

The OML Address tells OpenBSC the IP address of the BTS station. The BTS needs to know the IP address of the BSC in order to be able to establish an Abis/IP link to it. This is what's called the primary OML address. It is possible to set up with the following command:

Listing 31: Config: nanoBTS, set IP

```
1 ipaccess-config -o <BSC_IP> <BTS_IP>
```

After this the nanoBTS should be operational. A debugging communication with the BTS is possible via telnet. When setting the NVRAM flag 0x400, a special telnet console will be activated on TCP port 3210 of the nanoBTS. However, there is some kind of shared-secret challenge-response MD5 authentication required - otherwise the BTS will simply disconnect the user after a few seconds. To enable this feature execute:

Listing 32: Config: nanoBTS telnet

```
1 ipaccess-config -n 0x400/0x400 <BTS_IP>;
```

Then the user can login and configure the nanoBTS NVRAM settings by using the modified telnet client described in the telnet section. Connect with the following command:

Listing 33: Config: nanoBTS telnet

```
1 ipaccess-telnet <BTS_IP> 3210
```

2.7 Intermediate summary

This section provides a complete guide for the operation of a NiaB GSM network with local scope. The needed hardware and software components, their installation and configuration are comprehensively described. Different test were conducted in a laboratory environment to provide a first idea about the features and the stability of the setup. The next section will deal with the needed enhancement to provide a wide-area GSM network with public connectivity.

3 Concept of wide-area GSM NiaB with public connectivity

While the previous section focused on a local operation of the GSM network for example inside a building (i.e. a company from the private or public sector) the following will describe how to extend the current functionality to a wide-area GSM network. The purpose of such a wide-area GSM network can be GSM service provisioning for rural areas where no fixed-line connection to the internet is available (compare 2.1 and 3.1).

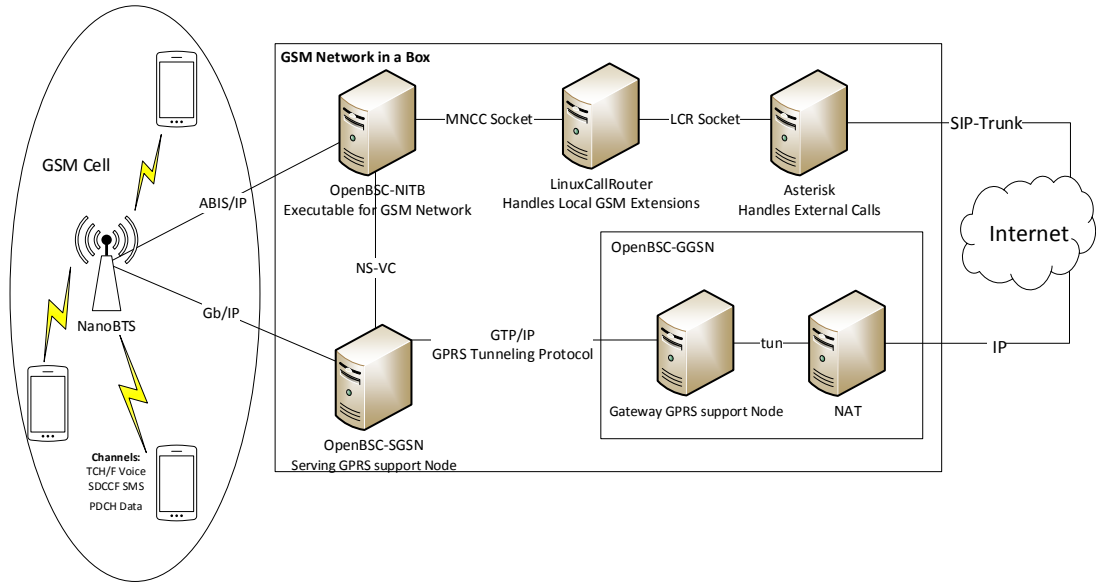


Figure 3.1: Concept of a GSM NiaB with public connectivity.

To provide public connectivity the following main issue need be addressed:

- Connection GSM NiaB to the Internet: The IP protocol is used to connect the NiB to a public network (Internet). This choice provides a huge interoperability since it is widely adapted by multiple ISP. To provide a cost effective solution for rural areas WiBACK is used. A low-cost self-managing wireless Backhaul solution to provide the connection to rural areas where the GSM should operate.

This issue will be addressed in the following subsections.

3.1 Internet interconnect using WiBACK

For a local operation the GSM Box will be placed at the nearest point with Internet connectivity. The connection of the nanoBTS can be realized using one or multiple radio links based on different technologies. A preferable choice is WiBACK which continues the concept of an easy to deploy network with low CAPEX and OPEX. A description of WiBACK is not provided in this document and the reader is pointed to the very comprehensive documentation available online in [Fok15].

3.2 Public connectivity

At this point the GSM Box is a closed system where a different cellphones can call each other and place test calls. To enhance the system to be available for calls from the outside a few changes have to be implemented in the Asterisk dialplan and the configuration of the Linux Call Router.

3.2.1 LCR configuration

First the interface configuration file of the LCR has to be changed so it is able to forward calls to the Asterisk server running on the same machine. Note that the interface name placeholder referring to listing 17 was changed from [Ext] to [AST]. This is not a necessary change but enhances the readability of the configuration because the Asterisk server will not only provide connectivity to an external communication network but also enables the support for various softphones configured on client PCs. The important change is the parameter for bridging the Asterisk interface to the OpenBSC GSM interface in the last line of listing 34. With this option the Asterisk is able to forward calls to the GSM network and the parameter *context btsctrl* refers to the extension in the Asterisk dialplan. With this context the Asterisk server is able to handle calls coming from the LCR directing them to the correct receiver.

Listing 34: Config: LCR /usr/local/etc/lcr/interface.conf

```
1 [AST]
2 remote asterisk
3 context btsctrl
4 extern
5 earlyb no
6 tones yes
7 bridge GSM
```

3.2.2 Asterisk configuration

For the purpose of testing the interconnection between the GSM network and subscribers handled by the Asterisk server a softphone client set up on a PC needs to be configured in the SIP configuration file. Note that this is a simple and heavily insecure implementation for testing purposes only. Parameters like *dtmfmode* and *callerid* are not necessary but nice to have and helping the call handling between both instances. The most important part is the context parameter therefor it refers to the extension in the Asterisk dialplan and is mainly responsible for reacting to calls from the softphone. Later on there will be another configuration for a sipgate account which will provide the connection to the public telephone network.

Listing 35: Config: Asterisk SIP Config /etc/asterisk/sip.conf

```
1 [1337]
2 type=friend
3 username=1337
```



```
4 secret=1234
5 dtmfmode=rfc2833
6 callerid="1337" <1337>
7 host=dynamic
8 context=from_intern
```

After this configuration the dialplan in the extension configuration file of the Asterisk server needs to be changed. According to listing 20 there has to be made quite a lot changes to the dialplan. The context of *btsctrl* stays the same but a complete case handling for incoming and outgoing calls for the Asterisk server have been made.

With the contexts *from_intern* and *from_extern* a basic rule based regulation exists which determines the behavior for different call scenarios. These contexts define different rules who is able to call whom in the network. Basically incoming calls from internal softphones are allowed to call everyone. But calls coming in via the to be implemented sipgate account are only allowed to connect to internal softphones or cellphones registered to the GSM network provided by the OpenBSC.

The context *to_intern* defines a direct call structure over the SIP protocol to softphones with four digit numbers starting with the number 1 like the softphone configured in listing 35. This is just for testing purposes and can be configured in other ways at will.

to_extern will be changed later on for the use of the sipgate account.

The most important change to the dialplan is the context *to_cellphones* since the old configuration from listing 20 didn't work for the numbers given to the cellphones by OpenBSC controller. Until now the cellphones were only able to call other cellphones and with the changes to the interface configuration of the LCR in listing 34 they are able to call softphones handled by the Asterisk server. The changes in the context *to_cellphones* now enables softphones to call in the other direction and makes sure that only numbers provided by the OpenBSC are allowed to be called. This refers to a five digit number starting with the number 2, 3 or 4 followed by any other numbers so the extension covers the same numbers 20000 to 49999 provided by the OpenBSC controller to the registered cellphones. Note that the interface handed over to the LCR is the Asterisk interface. Using the GSM interface will not establish a connection to any cellphone and will result in error messages in the Asterisk logfiles reporting that everyone is busy or congested.

Listing 36: Config: Asterisk SIP Config /etc/asterisk/extensions.conf

```
1 [btsctrl]
2 include => to_intern
3 include => to_extern
4 include => to_cellphones
5
6 [from_intern]
7 include => to_intern
8 include => to_extern
9 include => to_cellphones
10
11 [from_extern]
12 include => to_intern
13 include => to_cellphones
```

```
14
15 [to_intern]
16 exten => _1XXX,1,Dial(SIP/${EXTEN},30,r)
17 exten => _1XXX,n,Hangup()
18
19 [to_extern]
20 ;to be implemented with pending sipgate account
21
22 [to_cellphones]
23 exten => _[2-4]XXXX,1,GotoIf($[${CALLERID(name)} != ""]?4)
24 exten => _[2-4]XXXX,2,Set(CALLIDORIG=${CALLERID(num)})
25 exten => _[2-4]XXXX,3,Set(CALLERID(num)=02${CALLIDORIG})
26 exten => _[2-4]XXXX,4,Dial(LCR/AST/${EXTEN},30,r)
```

3.2.3 Sipgate configuration

As described in the previous chapters the connectivity to a public telephone network is realized via a Voice-over-IP (VoIP) service provider. There are several providers offering accounts for free. Those free accounts mostly have a very restricted functionality and provide the users with only one number. For a proof of concept a free account is sufficient but depending on the use-case it might be necessary to pay a monthly fee for additional services and functionality.

For the testing purpose a free account from Sipgate is used and charged with a small amount of money to place some testing calls. Sipgate provides their customers with an ID and a password for using the actual VoIP account. These account information need to be configured into the Asterisk server and the rule set of the dialplan needs also to be extended.

In the following listings the id of the Sipgate account is described by the placeholder *<SIP-ID>* and the associated password is referenced by *<SIP-password>*. Please make sure to replace the brackets also to assure the correct functionality.

Listing 37: Extended Config: Asterisk SIP Config /etc/asterisk/sip.conf with VoIP account information

```
1 [general]
2 ...
3 register => <SIP-ID>:<SIP-password>@sipgate.de/<SIP-ID>
4
5 [<SIP-ID>]
6 type=peer
7 username=<SIP-ID>
8 fromuser=<SIP-ID>
9 secret=<SIP-password>
10 host=sipgate.de
11 insecure=invite
12 caninvite=yes
13 canreinvite=no
14 nat=yes
15 qualify=yes
16 dtmfmode=rfc2833
17 context=from_extern
```

In order to guarantee the correct functionality the VoIP account needs to be connected with the Asterisk server. Therefore it has to be registered in the SIP configuration file in the general options segment. Subsequently the account information has to be provided in the same configuration file to allow incoming calls from the public telephone network. The VoIP account should be bound to an actual landline number which can be called from every other landline phone or cellphone. The main goal is to ensure all incoming calls on this number are accepted by the asterisk server and the caller is requested to enter the internal number he actually wants to call. The account is set up in the sip configuration file in a separate segment which can be identified by the account id. The most important part is the context parameter because the behavior of the Asterisk server will be defined in the extension configuration file with this context.

The next listing describes the changes in the asterisk extensions configuration file. The first part defines the behavior for any call coming from the outside. The Asterisk server accepts the call to the VoIP account number and plays a audio file with the request for a number to be dialed internally. The resulting input from the caller then is used to connect to a existing peer connected to the Asterisk server. The DISA (*Direct Inward System Access*) function is used to save the input and hands it over to the separate extension *restricted_call_forwarding*. This step is a security measure so that the incoming caller is only allowed to connect to peers of the Asterisk server. There are two possibilities for enhanced security regarding the DISA function. Either a password is generated and will be requested from the caller before enabling the input for the number to be called or the available extensions are limited to internal calls only like it is described. If these measures are not taken it might be possible to place an outgoing call to the public telephone network which will result in charging the VoIP account unwillingly. Therefore the extension *restricted_call_forwarding* ensures only internal connections which are generally free of charge.

Listing 38: Additional Config: Asterisk SIP Config /etc/asterisk/extensions.conf

```
1  [from_extern]
2  exten => <SIP-ID>,1,Answer()
3  exten => <SIP-ID>,n,Playback(vm-enter-num-to-call)
4  exten => <SIP-ID>,n,DISA(no-password,restricted_call_forwarding)
5  exten => <SIP-ID>,n,Hangup()
6
7  [restricted_call_forwarding]
8  include => to_intern
9  include => to_cellphones
10
11 [to_extern]
12 exten => _0X.,1,Set(CALLERID(num)=<SIP-ID>)
13 exten => _0X.,n,Dial(SIP/${EXTEN}@<SIP-ID>,r)
14 exten => _0X.,n,Hangup()
```

The second part of the listing defines the actions from the Asterisk server to be performed when a internal peer wants place a call to the public telephone network. The extension *to_extern* is used if a dialed number starts with the digit zero and is then handed over to the defined SIP account from the VoIP service provider.

With these changes to the Asterisk server it is now possible to reach internal peers from the outside and vice versa.

4 Evaluation in a lab environment

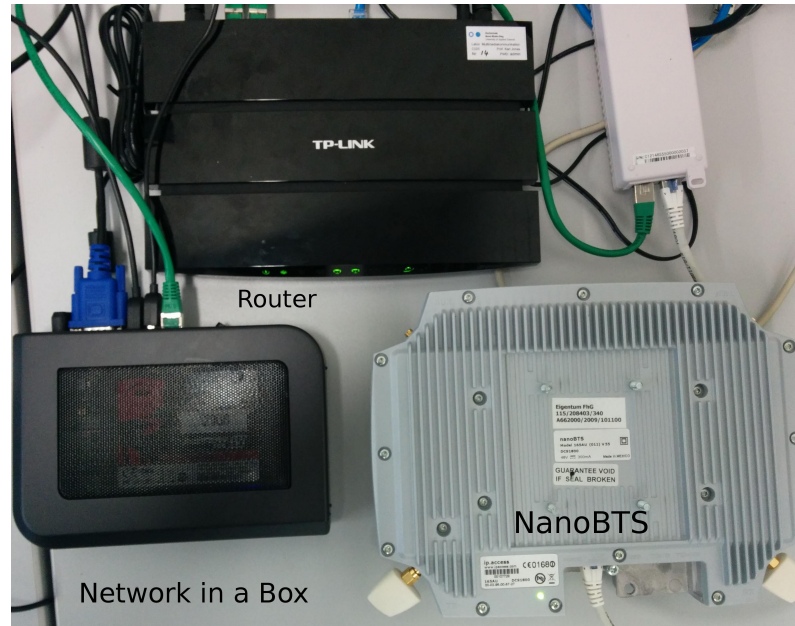


Figure 4.1: Evaluation in a lab environment

A basic evaluation of the described setup was conducted in a laboratory environment at the H-BRS (see figure 4.1). The previously described installation and configuration was conducted on a fan-less PC which can be seen as a prototype for the NiaB. An off-the-shelf router configures the basic network environment and connects the fan-less PC to the nanoBTS.

The needed configuration of the subscribers was conducted on the contemplated management interface visualized in figure 4.2.

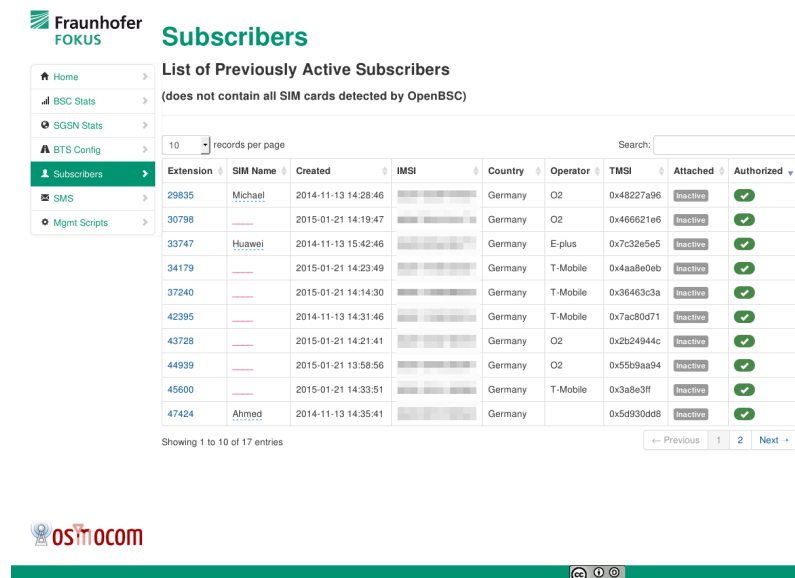


Figure 4.2: Management Interface for the GSM Subscribers

4.1 Compatible Clients

The first conducted test aims at evaluating the diversity of compatible clients with the nanoBTS. To do so, a number of different clients have been connected to the network and a test call has been conducted using the Asterisk echo-test feature. Table 1 provides an overview of the tested clients and the success rate.

Table 1: List of tested Clients in the laboratory environment

Vendor	Typ	OS	Version	ID	Working
HTC	Desire S	Android	4.04	rural	✓
LG	Nexus 5	Android	4.4	rural	✓
Samsung	Galaxy S3 mini	Android	4.1.2	rural	✓
Apple	Iphone 4s	iOS	8.1.2	rural	✓
Sony	Xperia SP	Android	4.3	rural	✓
HTC	legend	Android	4.1.2	rural	✓
Blackberry	Z10	OS	10.2.2.984	rural	✓
Mobile	Smart Phone A10	Android	4.04	rural	✓
Huawei	Y300	Android	4.1.1	rural	✓

At the moment all tested clients could successfully associate to the network and place the test call. This shows that the developed NiaB solution is able to operate with a wide range of different handsets. These results are important for a real-world build up where customers bring their own devices instead of being forced using particular hardware of an provider.

4.2 Number of simultaneously placed calls

In this section the maximum number of simultaneously placed calls will be evaluated. In the data sheet of the nanoBTS the maximum number is bounded to 16. However there seems to be some other limitations interfering with the maximum amount of active subscribers. During several tests only six to eight subscribers could be connected to the nanoBTS and place calls at the same time. Increasing the number of participants seems to stress the NiaB systems hardware so it freezes and all connections are dropped by the nanoBTS. This probably depends on the fan-less hardware architecture used in the setup and can easily avoided by replacing the system with a more powerful hardware constellation with an active cooling solution. However this has not been tested but regarding the heat generated by the fan-less system in this particular case this mostly had to be the reason. At one point of the experiments the system stopped working completely and needed to be returned to the manufacturer in order to be repaired.

4.3 Call setup success rate

In telecommunications, the call setup success rate (CSSR) is the fraction of the attempts to make a call that result in a connection to the dialed number (due to various reasons not all call attempts end with a connection to the dialed number). This is an important parameter when measuring customer satisfaction because a high number of unsuccessful call attempts is annoying and leads to additional busy times on the air interface.

Therefor several tests were conducted to verify the call setup success rate for different scenarios. In order to perform the different tests some preparations had to be made.

4.3.1 Softphone caller setup

At first a softphone needed to be set up on a local computer. During the tests an Apple MacBook from 2008 with an installed version of Mac OS X 10.10.3 combined with the softphone software *Telephone* in Version 1.1.4 was used. The system was directly connected via an Ethernet cable to one of the LAN ports of the OpenWRT Router the nanoBTS and the NiaB also were connected to. Thereby all systems were connected to the same subnet. The softphone was registered at the Asterisk server with the provided number.

Additionally a script was created with the Apple Mac OS X integrated software *Automator* which automatically places one hundred calls to the respective cellphone in an interval of 30 seconds.

To ensure the connection between both parties is established correctly the called cellphone must accept the calls placed by the softphone. This is described in the next chapter.

4.3.2 Cellphone callee setup

In the various scenarios the receiver of the test calls always has been a Huawei Y300 Android smartphone. Version 4.1.1 of the Android operating system was preinstalled on this phone.

To ensure every incoming call from a certain number is accepted automatically the Android software *Tasker* has been purchased from the *Google PlayStore*. With this app a automated workflow was created which accepts calls from the number of the softphone and the other cellphone from the next chapter and hang up after five seconds. Additionally a variable was created that is increased every time a call has been accepted. Thus the number of successfully accepted calls could be tracked and compared with the number of calls created by the caller. Ideally this counter stops at 100 at the end of an experiment.

Tasker allows the user to define different tasks which can be combined with so called *profiles* with whom the created task can be automatically executed. Therefor a task was created which access the phone functionality and takes the call. After a waiting period of five seconds the call is ended and eventually the call counter variable is increased by one.

This task is the reaction of a created profile and is executed in case a call is incoming on the smartphone from one of the two test caller numbers.

After each experiment the call counter variable needed to be reset to zero manually.

4.3.3 Cellphone caller setup

As a second caller device a HTC Desire Android based smartphone was used to verify the call success rate on a GSM only base. Therefore the Android application *Tasker* is also used to automatically call the other smartphone in a certain time frame. A variable similar to the *Tasker* variable on the receiving smartphone is used to control an outgoing call loop. The variable starting at the value of 100 is used in an if-statement. As long as the variable is greater than zero the following commands are executed. At first there is a waiting period of 5 seconds to be performed before the number of the receiving smartphone is dialed. After another waiting period of 15 seconds while the call is set up and hold the call eventually is terminated. Due to this waiting period the receiver of the call is able to recognize, accept and terminate the placed call to ensure the *Tasker* process on the receiving phone can be performed correctly. After the call is terminated the variable is decreased by one and the loop starts over again until the variable finally reaches the value zero.

To accomplish the process is automatically performed by *Tasker* a profile is needed. Unfortunately there is no incoming signal on this device like the incoming call on the receiving phone. So a profile has been created which reacts to receiving a text message from the receiver of the calls to place. The task is only executed when the number of the sender and the content of the text message is set properly.

After every experiment the variable is reset to 100 manually. Thereby in each experiment a maximum number of 100 calls are placed and ideally connected successfully.

4.3.4 Positions

To ensure the nanoBTS works correctly under different circumstances experiments were conducted with the clients placed in a variety of different positions. Due to these positions various signal strength values were reached and the impact of connections underlying a lesser signal quality could be monitored. There were six different positions tested during the experiments:

1. **Near BTS:** At this position the client is only a few centimeters apart from the nanoBTS station and received its signals with -59dB (96 %). In the experiments including the softphone this client was mostly placed at this position because of the necessity of a Ethernet connection to the gateway router.
2. **Hidden 1:** At this position the client was placed behind a computer display and so it received the signals with -73dB (71 %) from the nanoBTS. Also there were several

objects further in the line of sight (other computer displays and two persons). There was a total distance of approximately 4.5 meters between both devices and they were positioned at the same height.

3. **In cupboard:** This position was round about 7 meters apart from the nanoBTS and the client was placed inside of a cupboard for an increased attenuation. Nevertheless the signals were received with a value of -77dB (63 %) on a slightly elevated position according to the nanoBTS.
4. **Elevated:** The effect of an elevated position was further examined at this position. Therefore the client was placed onto a cupboard at a height of 2.5 meters and so about 1.5 meters above the nanoBTS. The distance to the nanoBTS was also about 7 meters, but a direct line of sight could be established to it. Thus the signals were received with a value of -73dB (71 %).
5. **Hidden 2:** The line of sight between the nanoBTS and the client was blocked by several objects (computer display and a person) at this position. The signals of the nanoBTS were still received with a value of -69dB (78 %). Both devices were approximately 4 meters apart of each other.
6. **Other room:** This position was by far the hardest to achieve because of the necessity of a connection quality that was as bad as possible but also can establish a stable connection to the GSM network. Therefore the client was placed in a different room connected through a inner courtyard on the same height of the nanoBTS. Both devices were roughly 20 meters apart and disturbed by two fronts of windows. The signals were received with -105dB (14 %).

These various positions were chosen to facilitate a heterogeneous set of scenarios for possible connections to the nanoBTS that could occur in a real life setting. Several obstacles, distances and heights between a client and the nanoBTS and between both clients as well were taken into account for the conducted experiments.

The results of these tests are depicted in the next chapter.

4.3.5 Results

Softphone to cellphone

The results of the miscellaneous experiments conducted between the stationary softphone and the mobile client in form of a smartphone are listed below in table 2.

These results clearly depict the reliability of the system. In all experiments every call was connected to its receiver. The overall amount of calls during an experiment was lowered step by step due to the fact that there seemed no necessity of such large sample spaces. However there were some minor issues during one experiment with the Asterisk server. At some point the Asterisk server refused all connections and the service needed to be restarted on the system. This issue might be associated with the large amount of placed

Table 2: Results of the conducted experiments between a softphone and a cellphone

No.	Sender position	Receiver position	Calls	CSSR (in %)
1	near BTS	near BTS	500	100
2	near BTS	hidden 1	400	100
3	near BTS	in cupboard	300	100
4	near BTS	elevated	300	100
5	near BTS	hidden 2	200	100
6	near BTS	other room	200	100

calls and the small waiting period between the calls. Maybe this is some kind of prevention mechanism for denial of service attacks. Further investigations regarding this issue were not pursued but could be taken in consideration to further enhance the reliability of the system.

Cellphone to cellphone

After concluding the experiments between the stationary softphone and the mobile client a variety of experiments with two mobile clients were performed. The results of these experiments are depicted below in table 3.

These results also show the reliability of the system. At first the calling client was positioned directly next to the nanoBTS like in the other series of experiments with the stationary softphone. These were the initial tests for a GSM-to-GSM-connection and after these proved to be very stable the positions were shifted differently. The following experiments were constructed to test different scenarios with difficult connection parameters. The position combinations chosen during these tests were mostly without a direct line of sight to the nanoBTS and between the two clients and also several tests were conducted with as bad as possible signal parameters like they were in a other room. Generally all tests could be performed without any bigger problems. It was difficult to find a good place at position 6 in the neighboring room so the connection was available all the time during the tests. Also there was the same issue with the refusing Asterisk server during one set of experiments. Generally the amount of calls during an experiment was lower than in the series before. This decision was based on the experiences already gathered by the other series and to decrease the amount of time in favor to test more variations of positions.

However the achieved results during all of these experiments prove the overall reliability of the system.

Table 3: Results of the conducted experiments between two cellphones

No.	Sender position	Receiver position	Calls	CSSR (in %)
1	near BTS	near BTS	300	100
2	near BTS	hidden 1	200	100
3	near BTS	in cupboard	200	100
4	near BTS	elevated	200	100
5	near BTS	hidden 2	200	100
6	near BTS	other room	100	100
7	hidden 1	Near BTS	100	100
8	hidden 1	hidden 2	200	100
9	hidden 1	other room	100	100
10	in cupboard	hidden 2	200	100
11	in cupboard	other room	100	100
12	elevated	hidden 2	200	100
13	elevated	near BTS	200	100
14	elevated	other room	100	100
15	hidden 2	near BTS	200	100
16	hidden 2	hidden 1	200	100
17	hidden 2	in cupboard	200	100
18	hidden 2	elevated	200	100
19	hidden 2	other room	100	100
20	other room	in cupboard	100	100

5 Conclusion

In this report we successfully described the installation, evaluation and usage of a GSM Network in a Box solution. This solution uses off-the-shelf hardware components as well as open-source software and provides therefore a costs-effective alternative compared to common base station technologies used by ISPs.

The GSM Network in a Box can be used to cellular networks in areas where no fixed-line connections are available. Using the provided guide-lines the solution runs on common hardware and we showed its reliability in a laboratory environment. Following the ICT4D concept we used an easy to use hardware and software combination which can be installed and maintained by non-specialized persons. This includes two major components. First, WiBACK as a cost-effective and self managing wireless-backhaul solution to provide network connectivity in rural areas. Second a web-interface to manage the OpenBTS GSM. For actual operation of the GSM network only a valid GSM license is needed. In our deployment and evaluation scenario we showed three important results. First a wide range of different cellular phones has been successfully tested. Second we show that a certain number of simultaneously placed calls is possible and that this parameter scales with the processing power of the machine running the GSM software stack. Third, we tested the stability and showed a very high call setup success rate in different scenarios. Overall we can conclude that the presented GSM Network in a Box is a possible solution to provide GSM network connectivity to rural areas.

5.1 Future Work

Future work should focus on the deployment in a larger testbed with additional user. For ISPs the usage of a Authentication Authorization Accounting (AAA) maybe desirable for example with prepaid cards or vouchers.

References

- [Eve14] A. Eversberg. (2014) Linuxcallrouter. [Online]. Available: <http://www.linux-call-router.de/>
- [Fok15] F. Fokus. (2015) Wireless backhaul technology fraunhofer fokus wiback. [Online]. Available: <http://www.wiback.org/>
- [tea13] T. D. I. team. (2013) Installing debian gnu/linux 7.0 for i386. [Online]. Available: <https://www.debian.org/releases/stable/i386/>
- [Wel10a] H. Welte. (2010) Openbsc changelog. [Online]. Available: <http://cgit.osmocom.org/openbsc/commit/?id=eafe22ca7290280fca0f8d31d70239fd0b0dbb9d>
- [Wel10b] ——. (2010) Openbsc project. [Online]. Available: http://openbsc.osmocom.org/trac/wiki/OpenBSC_LCR
- [Wel13] ——. (2013) osmo-nitb. [Online]. Available: <http://openbsc.osmocom.org/trac/wiki/osmo-nitb>
- [WF13a] H. Welte and H. Freyther. (2013) Osmocom openbsc. [Online]. Available: <http://openbsc.osmocom.org/trac/wiki/OpenBSC>
- [WF13b] ——. (2013) Osmocom openbsc. [Online]. Available: http://openbsc.osmocom.org/trac/wiki/GPRS_bitrates